PlayNAC KERNEL Codebase

> **Civilization-Scale Civic Simulation Engine for New Age Cybernetics** *The software expression of* $C = R \times P / M$

Show Image

Show Image

Show Image

Show Image

Author: Joseph A. Sprute (ERES Maestro) **Organization:** ERES Institute for New Age Cybernetics **Contact:** eresmaestro@gmail.com

---

About This Repository

This is the **authoritative home** of the PlayNAC KERNEL codebase — the software implementation layer of the ERES New Age Cybernetics framework. The KERNEL is an intelligent, voice-controlled civic simulation engine that translates the ERES Trilogy's theoretical architecture (BEST/SOUND/GOOD) into deployable code modules.

The current release is `PlayNAC_KERNEL_GitHub_v3.1.zip`, containing the complete Python codebase with 11 core modules.

> **Repository lineage:** This repo was originally named "ZIP" and served as a compressed archive store. It is being repurposed as the canonical codebase repository for PlayNAC KERNEL. Documentation archives remain in PlayNAC-KERNEL (PDFs) and Gracechain-Meritcoin (working documents).

What PlayNAC KERNEL Does

PlayNAC KERNEL is a **civic simulation engine** — not a game, not a chatbot, but a software system that models and facilitates civilization-scale governance, economics, and resource coordination through gamified interaction. It implements the ERES Trilogy's three-tier architecture in code:

```
BEST (Personal)          →  EarnedPath, CARE module, Vacationomics
SOUND (Public-Private)   →  SOMT recorder, JAS consensus, VERTECA voice nav
GOOD (Verification)      →  Bio-resonance validation, GERP media processing,
                            blockchain attestation, 4D visualization
```

The system is designed to operate as a **Ship's Computer** — a continuous-loop AI that processes voice commands, routes civic intent, validates contributions through resonance metrics, and records state on-chain. The "ship" metaphor is literal at maximum scale: the VLSA Principle states *"If it works in a THOW, it works on a generation ship."* The KERNEL is the software that runs on both.

Core Modules

System Architecture

```
                    ┌──────────────────────┐
                    │   main_ship_ai.py    │
                    │  (Continuous Loop)   │
                    └──────────┬───────────┘
                               │
                    ┌──────────▼───────────┐
                    │ voice_nav_module.py  │
                    │    (VERTECA Input)   │
                    └──────────┬───────────┘
                               │
                    ┌──────────▼───────────┐
                    │  kernel_router.py    │
                    │ (Intent Dispatcher)  │
```

```
                    └──┬──┴──┬──┴──┬──┘
                       │     │     │
            ┌──────────▼┐  ┌▼─────▼──┐  ┌▼────────────┐
            |care_module| |media_    | |jas_graph.py |
            |.py (CARE) | |kernel.py | |(Consensus)  |
            └───────────┘ |(GERP)    | └─────────────┘
                          └──┬──────┘
                             |
                   ┌─────────▼──────────┐
                   |    bio_pow.py      |
                   |(Resonance Validate) |
                   └─────────┬──────────┘
                             |
            ┌────────────────┼────────────────┐
            |                |                |
   ┌────────▼──┐   ┌─────────▼─┐   ┌─────────▼──┐
   |somt_      |   |geo_       |   |4d_visual_  |
   |recorder.py |   |perspective |   |env.py      |
   |(State Log) |   |.py (Geo)  |   |(Three.js)  |
   └───────────┘   └───────────┘   └────────────┘
```

Module Reference

Module	Role	Trilogy Tier	Description
`main_ship_ai.py`	**Entry point**	All	Continuous-loop Ship's Computer — orchestrates all subsystems, processes voice input, maintains session state
`voice_nav_module.py`	**Input**	SOUND	VERTECA hands-free voice navigation — speech recognition for civic intent routing using 4D spatial audio model
`kernel_router.py`	**Dispatch**	SOUND	Intent dispatcher — routes parsed voice commands to appropriate subsystem (GERP, PlayNAC, EarnedPath, CARE)
`playnac_kernel.py`	**Orchestrator**	BEST	Main PlayNAC engine — coordinates media tasks, resonance validation, and blockchain attestation
`care_module.py`	**Governance**	BEST	CARE (Choice, Action, Response, Evaluation) scoring logic — civic decision modeling for

Module	Role	Trilogy Tier	Description
			resource domains (Water, Immigration, Security)
`media_kernel.py`	**Processing**	GOOD	GERP visual transformations and Markdown complexity validation — processes media state for resonance assessment
`bio_pow.py`	**Validation**	GOOD	Bioenergetic resonance validation engine — validates contributions through entropy-based coherence measurement aligned with BERA metrics (ARI, ERI, RHC, RCI)
`jas_graph.py`	**Consensus**	SOUND	JAS Link-based task consensus and validation — decentralized agreement protocol for civic task completion
`somt_recorder.py`	**State**	GOOD	SOMT (Sociocratic Overlay Metadata Tapestry) state recorder — logs system state for provenance tracking and sustainability auditing
`geo_perspective.py`	**Spatial**	GOOD	Geographic-spiritual alignment encoder — maps civic activity to spatial coordinates for GAIA infrastructure integration
`4d_visual_env.py`	**Interface**	BEST	Web-based 4D simulation interface using Three.js — visualization layer for civic simulation state and VERTECA spatial navigation

System Workflow

```
1. VOICE INPUT        User speaks a civic command ("show water usage")
                      |
2. SPEECH PARSE       voice_nav_module.py processes audio → intent
                      |
3. INTENT ROUTE       kernel_router.py dispatches to target module
                      |
4. MODULE EXECUTE     care_module.py / media_kernel.py / playnac_kernel.py
                      |
```

```
5. RESONANCE CHECK    bio_pow.py validates contribution against BERA metrics
                      |
6. CONSENSUS          jas_graph.py records task on distributed ledger
                      |
7. STATE LOG          somt_recorder.py writes to SOMT provenance record
                      |
8. VISUALIZE          4d_visual_env.py renders updated state in Three.js
```

This workflow implements C = R × P / M in software:

- **C (Cybernetics)** = the KERNEL's orchestration loop
- **R (Resource)** = civic data, media state, geographic coordinates, voice input
- **P (Purpose)** = CARE scoring, EarnedPath progression, resonance validation
- **M (Method)** = VERTECA routing, JAS consensus, SOMT logging, blockchain attestation

---

VERTECA Voice Commands

The Ship's Computer responds to natural-language civic commands routed through VERTECA:

Command	Action	Target Module
`"Start game"`	Launch PlayNAC civic simulation	`playnac_kernel.py`
`"Learn path"`	Open EarnedPath training modules	`kernel_router.py` → EarnedPath
`"Show water"`	Fetch water resource metrics from GERP	`media_kernel.py`
`"Record state"`	Log current CARE state into SOMT	`somt_recorder.py`
`"What is CARE?"`	Return system description	`care_module.py`
`"Check resonance"`	Run BERA validation on current state	`bio_pow.py`
`"Show map"`	Render geographic perspective in 4D	`geo_perspective.py` → `4d_visual_env.py`

Key Formulas Implemented in Code

EarnedPath Progression

```python
# EP = CPM × WBS + PERT
earned_path = critical_path_method * work_breakdown_structure + program_eval_review_
```

CARE Scoring

```python
# Choice, Action, Response, Evaluation
care_score = evaluate(choice_weight, action_impact, response_quality, evaluation_fee
```

Resonance Validation (BERA-aligned)

```python
# Bioenergetic coherence measurement
# Aligned with ARI, ERI, RHC, RCI
resonance_valid = entropy_coherence >= threshold
# Proof-of-Resonance: harmony IN → value OUT
```

Vacationomics

```python
# Work-leisure balance rooted in Golden Ratio
vacationomics_score = somt_score * berc_score * (eri / ari)
# φ = 0.618... — nature's optimization constant
```

Quick Start

```bash
```

```
# Clone this repository
git clone https://github.com/ERES-Institute-for-New-Age-Cybernetics/ZIP.git
cd ZIP

# Extract the codebase
unzip PlayNAC_KERNEL_GitHub_v3.1.zip -d PlayNAC_KERNEL
cd PlayNAC_KERNEL

# Install dependencies
pip install -r requirements.txt

# Launch the Ship's Computer
python main_ship_ai.py
```

Requirements

- Python 3.10+
- Dependencies specified in `requirements.txt` within the archive
- Optional: microphone access for VERTECA voice navigation
- Optional: Three.js-compatible browser for 4D visualization

Relationship to the ERES Trilogy

The PlayNAC KERNEL is the **software proof** of the Trilogy's theoretical claims. Each book's instruments have corresponding code:

Trilogy Book	Instruments	Code Modules
Book 1: One Good (BEST)	UBIMIA, EarnedPath, PlayNAC, Meritcoin	`playnac_kernel.py`, `care_module.py`, `4d_visual_env.py`
Book 2: Security-Clearance (SOUND)	IDIPITIS, GSSG, SOMT, VERTECA	`voice_nav_module.py`, `kernel_router.py`, `jas_graph.py`
Book 3: Data-Integrity (GOOD)	FAVORS, CBGMODD, BERA, GAIA-SOMT	`bio_pow.py`, `media_kernel.py`, `somt_recorder.py`, `geo_perspective.py`

The VLSA Principle applies directly: the same KERNEL that runs a civic simulation on a laptop can, at maximum scale, run the Ship's Computer on an FDRV generation ship. The code is the

scale-proof.

Relationship to Other ERES Repositories

Repository	Role
ZIP (this repo)	Authoritative codebase — deployable Python modules
PlayNAC-KERNEL	Comprehensive PDF research archive (216+ docs) — theory behind the code
Gracechain-Meritcoin	Active working archive — includes `ERESPlayNACKERNELCodebaseV1.zip` (earlier version), VERTECA connector code, and framework specifications
Proof-of-Work_MD	Markdown specifications — protocol definitions that the code implements
Support-Documentation	Packaged R&D — white papers, legal instruments

Codebase Version History

Version	Location	Notes
v3.1 (current)	This repo: `PlayNAC_KERNEL_GitHub_v3.1.zip`	11 modules, Ship's Computer architecture
v1.0	Gracechain-Meritcoin: `ERESPlayNACKERNELCodebaseV1.zip`	Initial codebase release
Grok v3	Proof-of-Work: `ERES PlayNAC Grok KERNEL Codebase V.3.pdf`	Grok LLM collaboration documentation

---

Development Roadmap

Current (v3.1)

- 11 core Python modules
- VERTECA voice navigation

- CARE civic scoring
- JAS decentralized consensus
- SOMT state recording
- Three.js 4D visualization
- Bio-resonance validation

Planned

- [ ] **BERA-PY library** — standalone Python library for BERA resonance metrics (ARI, ERI, RHC, RCI)
- [ ] **SCALULAR integration** — certification pillar endpoints (HEALTH, LAW, PROTECTION, SKILLS/TRADE)
- [ ] **FAVORS adapter** — fault-tolerant data validation layer
- [ ] **Meritcoin bridge** — Proof-of-Resonance token issuance from validated contributions
- [ ] **GraceChain connector** — distributed trust ledger integration
- [ ] **VLSA test harness** — automated scale-proof testing (targeting 91/91 coverage parity with SPT Papers)
- [ ] **IDIPITIS identity module** — identity-without-surveillance for user authentication
- [ ] **SROC contract engine** — Smart-Resonant Offset Contract execution

---

Architecture Principles

The PlayNAC KERNEL follows four engineering principles derived from the ERES framework:

1. Moral vectoring as runtime constraint. Every module enforces BEST/SOUND/GOOD outcomes — the ethical constraint *"Don't hurt yourself. Don't hurt others. Build for generations to come"* is not aspirational commentary, it is a validation gate in `bio_pow.py` and `care_module.py`.

2. Proof-of-Resonance, not Proof-of-Work. The `bio_pow.py` module validates contributions through bioenergetic coherence measurement aligned with BERA metrics — value accrues from harmony, not from computational waste. The four BERA indices (ARI, ERI, RHC, RCI) feed the validation loop.

3. Ship's Computer as design metaphor. `main_ship_ai.py` runs as a continuous loop because the system it models — a civilization — never stops. The VERTECA voice interface treats the user as crew, not as customer. The Ship's Computer serves the mission; the mission is $C = R \times P / M$.

4. VLSA compliance. Every module must function identically at THOW scale (a single user on a laptop) and at FDRV scale (a generation ship with thousands of crew). The code makes no

assumptions about deployment context that would break at either extreme.

Contributing

The PlayNAC KERNEL is open to contributions in the following areas:

Area	Needs
Module development	BERA-PY library, FAVORS adapter, SCALULAR endpoints
Testing	Unit tests, integration tests, VLSA scale-proof harness
Voice/NLP	VERTECA command expansion, multilingual support
Visualization	Three.js 4D environment enhancements
Documentation	Module-level docstrings, API documentation, tutorials
Security	IDIPITIS identity module, input validation, privacy review

All contributions must align with CCAL v2.1 licensing and the ERES ethical constraint.

Canonical Terminology

This codebase uses ERES canonical terminology. Key corrections from earlier documentation:

Term	Canonical Expansion	Common Error
SOMT	Sociocratic Overlay Metadata Tapestry	~~Strategic Optimization & Merit Tracking~~
SROC	Smart-Resonant Offset Contracts	~~Smart Registered Offset Contracts~~
UBIMIA	Universal Basic Infrastructure, Meritcoin Incentivized Architecture	~~Universal Basic Income + Merit + Incentives + Awards~~
RHC	Resonant Harmony Cycle	~~Resonant Harmony Cybernetics~~
Validation	Proof-of-Resonance	~~Proof-of-Work~~

License

All original ERES code and documentation is published under the **CARE Commons Attribution License v2.1 (CCAL)**.

- **Permitted:** Civic, educational, and research use; open-source derivative works with attribution; free redistribution
- **Prohibited:** Weaponization; extractive commercial exploitation; closed-source derivatives

> **Note:** Previous versions of this README referenced "Creative Commons BY-NC 4.0." The canonical license for all ERES work is CCAL v2.1.

Author

Joseph Allen Sprute (ERES Maestro) Founder and Director, ERES Institute for New Age Cybernetics U.S. Army Veteran — Oregon Army National Guard, 1983–1989, Infantry 11B, Honorable Discharge 33 Westbury Drive, Bella Vista (Beautiful View), Arkansas 72714
eresmaestro@gmail.com

Co-Author: SYU JIA WUN (independent researcher, Taiwan)

> *"If it works in a THOW, it works on a generation ship."* The KERNEL is the software that runs on both.

Document ID: ERES-PLAYNAC-KERNEL-CODEBASE-README-2026-001 **Last Updated:** March 28, 2026 **Classification:** Open / Unrestricted